

# Architecture and Security Note

Auth, Webhooks, and Data Handling

Esusu Engineering Team

July 7, 2026

## Contents

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>System Overview</b>                 | <b>2</b> |
| <b>2</b> | <b>Authentication</b>                  | <b>2</b> |
| 2.1      | Registration Flow . . . . .            | 2        |
| 2.2      | JWT Tokens . . . . .                   | 2        |
| 2.3      | Password Hashing . . . . .             | 2        |
| 2.4      | Account Lockout . . . . .              | 2        |
| 2.5      | Transaction PIN . . . . .              | 2        |
| 2.6      | API Route Protection . . . . .         | 2        |
| <b>3</b> | <b>Webhook Security</b>                | <b>3</b> |
| 3.1      | Endpoint . . . . .                     | 3        |
| 3.2      | Signature Verification . . . . .       | 3        |
| 3.3      | Timestamp Validation . . . . .         | 3        |
| 3.4      | Idempotency / Deduplication . . . . .  | 3        |
| 3.5      | Processing Flow . . . . .              | 3        |
| 3.6      | Supported Events . . . . .             | 4        |
| <b>4</b> | <b>Payment Reconciliation</b>          | <b>4</b> |
| 4.1      | Payment Classification . . . . .       | 4        |
| 4.2      | Reconciliation Process . . . . .       | 4        |
| <b>5</b> | <b>Data Handling</b>                   | <b>4</b> |
| 5.1      | Database Schema . . . . .              | 4        |
| 5.2      | Encryption at Rest . . . . .           | 4        |
| 5.3      | Input Validation . . . . .             | 5        |
| 5.4      | Database Connection . . . . .          | 5        |
| 5.5      | API Communication with Nomba . . . . . | 5        |
| <b>6</b> | <b>Security Considerations</b>         | <b>6</b> |
| 6.1      | Threat Mitigations . . . . .           | 6        |
| 6.2      | Environment Variables . . . . .        | 6        |
| 6.3      | Known Limitations . . . . .            | 6        |
| <b>7</b> | <b>Architecture Diagram</b>            | <b>7</b> |
| <b>8</b> | <b>Conclusion</b>                      | <b>7</b> |

## 1 System Overview

Esusu is a rotational savings (“esusu”) platform built on:

- **Backend:** Next.js 15 App Router (TypeScript), Drizzle ORM, PostgreSQL (Supabase)
- **Payments:** Nomba API (virtual accounts, transfers, webhooks)
- **Frontend:** Next.js with Zustand state management
- **Deployment:** Vercel (serverless functions)

The system uses a dedicated Nomba sub-account (49d53c87-e6e4-4c5f-92c6-f13d2579bece) to scope all payment operations. Each user receives a unique virtual account (VA) under this sub-account for collecting contributions.

## 2 Authentication

### 2.1 Registration Flow

1. User submits email, receives a 6-digit OTP via SMTP (Gmail).
2. OTP is stored in `otp_codes` with a 5-minute expiry.
3. On verification, a bcrypt-hashed password (12 rounds) is saved.
4. A Nomba virtual account is provisioned under the sub-account for the user.
5. A JWT is issued and returned to the client.

### 2.2 JWT Tokens

- **Algorithm:** HMAC-SHA256 (`jsonwebtoken` library).
- **Payload:** { `userId`, `email` }.
- **Expiry:** Configurable via `JWT_EXPIRES_IN` (default 7d).
- **Storage:** Client-side (Zustand persist + cookie for server-side middleware guard).

### 2.3 Password Hashing

- **Algorithm:** bcrypt with 12 salt rounds (`bcryptjs`).
- Never stored in plaintext. Verified via `bcrypt.compare`.

### 2.4 Account Lockout

Progressive lockout on failed login attempts:

| Failed Attempts | Lockout Duration |
|-----------------|------------------|
| 5               | 15 minutes       |
| 10              | 1 hour           |
| 15              | 1 year           |

### 2.5 Transaction PIN

A 4-digit PIN (hashed with bcrypt) is required for financial operations (contributions, withdrawals). Verified via `/api/v1/auth/verify-pin`.

### 2.6 API Route Protection

All protected routes call `requireAuth(request)` which:

1. Extracts the `Authorization: Bearer <token>` header.

2. Calls `verifyToken()` to decode and validate the JWT.
3. Returns `401 Unauthorized` if invalid or missing.

There is no Next.js middleware file; auth is enforced at the route handler level.

## 3 Webhook Security

### 3.1 Endpoint

```
POST /api/v1/webhooks/nomba
```

Publicly accessible (no JWT auth required). Security is ensured via HMAC signature verification.

### 3.2 Signature Verification

Nomba signs every webhook payload using a shared secret (`NOMBA_WEBHOOK_SECRET`). The verification process:

1. Construct the signing string by concatenating 9 colon-separated fields:

```
event_type:requestId:userId:walletId:
transactionId:type:time:responseCode:timestamp
```

2. Compute `HMAC-SHA256(secret, signing_string)` and Base64-encode.
3. Compare with the `nomba-signature` header using `crypto.timingSafeEqual` (constant-time comparison to prevent timing attacks).
4. Reject with `401` on mismatch.

### 3.3 Timestamp Validation

- Parses the `nomba-timestamp` header (RFC-3339 or epoch ms).
- Rejects if the timestamp is older than **1 hour** (to match Nomba's retry window of up to 53 minutes across 5 retries).
- Prevents replay attacks from stale payloads.

### 3.4 Idempotency / Deduplication

- Before processing, checks `webhook_events` for an existing record with the same `nomba_request_id`.
- Returns `"duplicate"` immediately if found (synchronous check).
- Inserts the raw payload into `webhook_events` with status `"received"` before processing.

### 3.5 Processing Flow

1. Synchronous: signature check → timestamp check → dedup check → DB insert → return `200 Received`.
2. Asynchronous (fire-and-forget): `processWebhook()` dispatches to the appropriate handler based on `event_type`.
3. Status lifecycle: `received` → `processed` / `failed` / `unhandled`.

**Note:** The fire-and-forget pattern means the `200` is returned immediately. In Vercel serverless, the function may terminate before processing completes. This is acceptable because the record is already persisted synchronously.

| Event            | Action                          |
|------------------|---------------------------------|
| payment_success  | Reconcile against contributions |
| payout_success   | Mark payout cycle as paid       |
| payout_failed    | Revert cycle to settling        |
| payment_failed   | Logged, no action               |
| payment_reversal | Logged, no action               |
| payout_refund    | Logged, no action               |

### 3.6 Supported Events

## 4 Payment Reconciliation

### 4.1 Payment Classification

Incoming payments are classified based on the ratio of actual vs. expected amount:

| Classification | Ratio Range              |
|----------------|--------------------------|
| Exact          | $0.99 \leq r \leq 1.01$  |
| Underpayment   | $0.50 \leq r < 0.99$     |
| Overpayment    | $1.01 < r \leq 1.50$     |
| Misdirected    | $r < 0.50$ or $r > 1.50$ |

### 4.2 Reconciliation Process

1. Match the incoming VA number (`aliasAccountNumber`) to a user's virtual account in the database.
2. If no match, route to orphan handling.
3. Look up the user's active circle memberships and expected contribution per circle.
4. Classify the payment and apply accordingly:
  - **Exact:** Create a contribution record, update cycle totals, check for cycle completion.
  - **Underpayment:** Create a partial contribution; the remainder becomes a debt.
  - **Overpayment:** Apply the expected amount; excess is logged and may be credited to the user's wallet.
  - **Misdirected:** Flag for manual review.
5. On cycle completion, trigger payout to the recipient member.

## 5 Data Handling

### 5.1 Database Schema

16 PostgreSQL tables managed via Drizzle ORM:

### 5.2 Encryption at Rest

- **Passwords:** bcrypt-hashed (12 rounds). Never stored in plaintext.
- **PINs:** bcrypt-hashed. Never stored in plaintext.
- **JWT Secret:** Stored in environment variables, never committed to source control.

| Table                            | Purpose                                    |
|----------------------------------|--------------------------------------------|
| <code>users</code>               | User accounts, credentials                 |
| <code>circles</code>             | Savings circles                            |
| <code>members_circles</code>     | Circle membership and rotation order       |
| <code>virtual_accounts</code>    | Per-user Nomba VAs                         |
| <code>cycles</code>              | Individual savings cycles                  |
| <code>contributions</code>       | Payment records per member per cycle       |
| <code>debts</code>               | Unpaid/late contribution tracking          |
| <code>otp_codes</code>           | Email OTPs for registration                |
| <code>webhook_events</code>      | Raw webhook payloads and processing status |
| <code>wallet_transactions</code> | Internal wallet ledger                     |
| <code>payout_transactions</code> | Bank transfer records                      |
| <code>notifications</code>       | In-app notifications                       |
| <code>invite_codes</code>        | Circle invitation codes                    |
| <code>referrals</code>           | User referral tracking                     |
| <code>user_settings</code>       | Per-user notification preferences          |
| <code>contact_messages</code>    | Support/contact form submissions           |

### 5.3 Input Validation

All API endpoints validate required fields and return structured error responses via the `ApiResponse` format:

```
{
  "code": "00" | "01" | "03" | "04" | "05",
  "description": "Success" | "Error message",
  "data": <payload> | null
}
```

Response codes:

- 00 — Success
- 01 — Bad request / validation error
- 03 — Unauthorized
- 04 — Not found
- 05 — Conflict (e.g. duplicate)

### 5.4 Database Connection

- Connection pool: max 3 connections, 30s idle timeout, 15s connect timeout.
- Singleton pattern via `globalThis` to survive hot reloads in development.
- All queries use Drizzle ORM (parameterized) — no raw SQL interpolation, preventing SQL injection.

### 5.5 API Communication with Nomba

- Authenticated via OAuth2 client-credentials flow (`POST /v1/auth/token/issue`).
- Tokens cached for 55 minutes in memory.
- The `accountId` header always carries the **parent account ID**. The sub-account ID is passed only as a path parameter (e.g. `/v1/accounts/virtual/{subAccountId}`).
- Idempotency keys (`X-Idempotent-key` header) are used for transfer operations to prevent duplicate transactions.

## 6 Security Considerations

### 6.1 Threat Mitigations

| Threat              | Mitigation                                                                   |
|---------------------|------------------------------------------------------------------------------|
| Webhook forgery     | HMAC-SHA256 signature verification with constant-time comparison             |
| Replay attacks      | 1-hour timestamp window + idempotent dedup via <code>nomba_request_id</code> |
| Brute force login   | Progressive account lockout (5/10/15 attempts)                               |
| Password theft      | bcrypt with 12 salt rounds                                                   |
| SQL injection       | Drizzle ORM parameterized queries                                            |
| Timing attacks      | <code>crypto.timingSafeEqual</code> for signature comparison                 |
| Duplicate transfers | Idempotency keys on outbound Nomba API calls                                 |

### 6.2 Environment Variables

Sensitive values are stored in environment variables and never committed:

| Variable                                        | Purpose                           |
|-------------------------------------------------|-----------------------------------|
| <code>DATABASE_URL</code>                       | PostgreSQL connection string      |
| <code>JWT_SECRET</code>                         | HMAC signing key for JWTs         |
| <code>JWT_EXPIRES_IN</code>                     | Token lifetime                    |
| <code>NOMBA_WEBHOOK_SECRET</code>               | HMAC key for webhook verification |
| <code>NOMBA_LIVE_CLIENT_ID</code>               | Nomba API client ID               |
| <code>NOMBA_LIVE_PRIVATE_KEY</code>             | Nomba API secret                  |
| <code>NOMBA_SUB_ACCOUNT_ID</code>               | Sub-account for scoping VAs       |
| <code>SMTP_USER</code> / <code>SMTP_PASS</code> | Email delivery credentials        |

### 6.3 Known Limitations

1. **Fire-and-forget webhooks:** Processing runs asynchronously. If the Vercel function terminates before `processWebhook` completes, the reconciliation may not execute. Mitigated by the synchronous DB insert, which allows manual or cron-based retry.
2. **No refresh token rotation:** Refresh tokens use the same signing key and expiry as access tokens. A stolen refresh token grants long-lived access.
3. **OTP whitelist:** Development mode uses a hardcoded phone number whitelist with OTP 999999. Must be removed before production.
4. **No rate limiting:** API routes do not enforce rate limits. Consider adding middleware for production.
5. **No CSRF protection:** JWT is sent via `Authorization` header, which provides some protection, but cookie-based auth should include CSRF tokens.

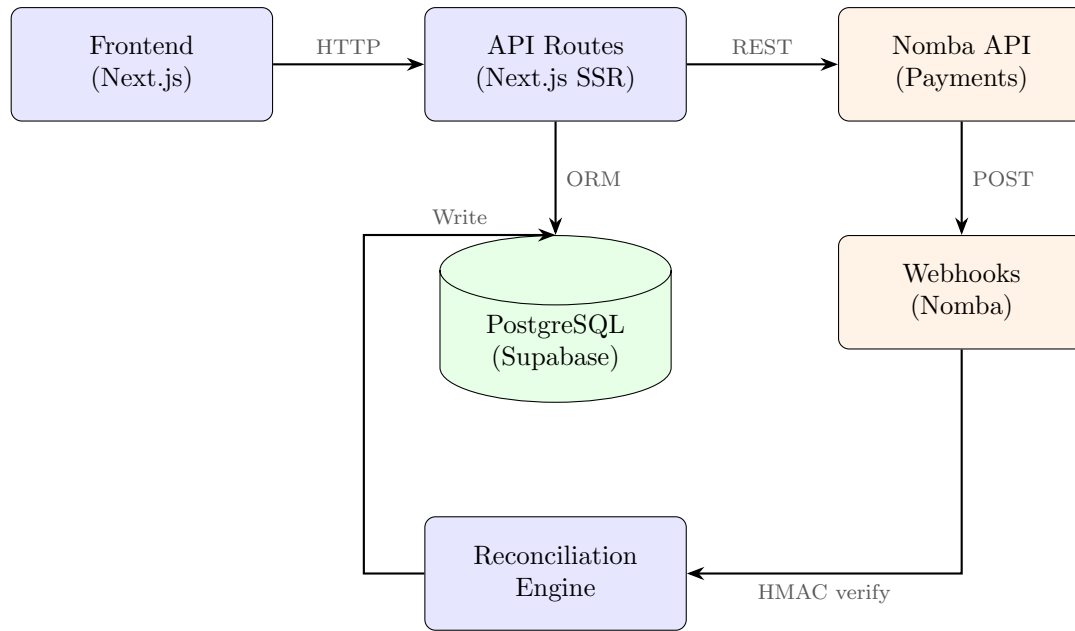


Figure 1: System architecture showing the request flow from frontend through API to Nomba, with webhook-based reconciliation loop.

## 7 Architecture Diagram

## 8 Conclusion

The Esusu platform implements defense-in-depth across authentication, webhook processing, and data handling. Key security properties:

- All sensitive data (passwords, PINs) is hashed at rest.
- Webhook authenticity is verified via HMAC-SHA256 with constant-time comparison.
- Financial operations require both JWT authentication and PIN verification.
- Idempotency is enforced at both the webhook ingestion layer and outbound Nomba API calls.